Revisão Essencial de Java: Arrays, Interfaces, Generics, Testes e Recursão

Bem-vindo a esta jornada de revisão dos conceitos fundamentais de Java! Esta apresentação cobre os pilares essenciais da programação Java moderna, desde a estruturação de dados com arrays até técnicas avançadas de testes e recursão. Seja você um estudante revisando conceitos ou um desenvolvedor reforçando fundamentos, esta apresentação oferece uma visão abrangente e prática dos tópicos mais importantes.

Java continua sendo uma das linguagens mais utilizadas no mundo, e dominar seus conceitos fundamentais é essencial para qualquer desenvolvedor. Vamos explorar como arrays e interfaces formam a base da estruturação de dados, como generics promovem código reutilizável e seguro, como testes garantem qualidade, e como recursão oferece soluções elegantes para problemas complexos.

Made with GAMMA

Arrays e Interfaces: Os Blocos de Construção

Arrays e interfaces são fundamentais para estruturar código Java eficiente e flexível. Vamos explorar cada um deles em detalhes:

Arrays: Estrutura de Dados Básica

Arrays são estruturas de dados que agrupam múltiplos elementos do mesmo tipo em posições de memória contíguas. Como objetos alocados no heap, eles são indexados de 0 ao tamanho-1, permitindo acesso rápido a qualquer elemento através de seu índice.

Sintaxe Básica:

```
int[] numeros = new int[5];
String[] nomes = {"João", "Maria", "Pedro"};
```

Vantagens:

- Acesso direto aos elementos via índice (O(1))
- Memória contígua otimiza cache da CPU
- Tipo homogêneo garante consistência
- Sintaxe simples e direta

Desvantagens:

- Tamanho fixo após criação
- Dificuldade em encontrar elementos sem índice
- Não suporta métodos de manipulação avançada
- Requer casting manual para tipos primitivos

Interfaces: Contratos Polimórficos

Interfaces definem contratos que classes devem seguir, especificando métodos que devem ser implementados sem fornecer a implementação real. Elas são fundamentais para o polimorfismo e desacoplamento de código.

Exemplo Prático:

```
interface Forma {
    double calcularArea();
    double calcularPerimetro();
}

class Circulo implements Forma {
    private double raio;

    public Circulo(double raio) {
        this.raio = raio;
    }

    @Override
    public double calcularArea() {
        return Math.PI * raio * raio;
    }

    @Override
    public double calcularPerimetro() {
        return 2 * Math.PI * raio;
    }
}
```

Vantagens:

- Promove polimorfismo e flexibilidade
- Desacoplamento entre contrato e implementação
- Suporte a múltipla herança de tipos
- Facilita testes e mock objects

Arrays: Pontos-Chave

- Todos os elementos devem ser do mesmo tipo
- Tamanho é imutável após criação
- Índices começam em 0
- Podem armazenar tipos primitivos e objetos
- São objetos (herdam de Object)
- Exceção ArrayIndexOutOfBoundsException para acesso inválido

Interfaces: Boas Práticas

- Use para definir comportamentos comuns
- Nome deve ser descritivo e usar verbo
- Evite implementação de métodos (antes do Java 8)
- Use default methods para funcionalidades opcionais
- Permita que classes implementem múltiplas interfaces

Made with GAMMA

Generics: Código Reutilizável e Seguro

Generics são um dos recursos mais poderosos do Java, permitindo criar classes, interfaces e métodos que operam em tipos especificados em tempo de compilação, garantindo segurança de tipo e eliminando a necessidade de casting manual.

01

Segurança de Tipo em Tempo de Compilação

Generics garantem que erros de tipo sejam detectados durante a compilação, não em tempo de execução. Isso elimina `ClassCastException` e torna o código mais robusto.

02

Eliminação de Casting Manual

Sem generics, você precisaria fazer casting em cada acesso ao elemento. Com generics, o compilador gerencia isso automaticamente, tornando o código mais limpo e seguro.

03

Reutilização de Código

Uma única implementação de classe ou método funciona com múltiplos tipos, reduzindo duplicação de código e aumentando a manutenibilidade.

Exemplos Práticos de Generics

ArrayList com Generics

```
ArrayList nomes = new ArrayList<>();
nomes.add("João");
nomes.add("Maria");
nomes.add("Pedro");

// Acesso seguro sem casting
String primeiroNome = nomes.get(0);
System.out.println(primeiroNome); // João

// Erro de compilação se tentar adicionar tipo errado
// nomes.add(123); // Não compila!
```

Classe Genérica Personalizada

```
public class Caixa {
    private T conteudo;

    public void guardar(T item) {
        this.conteudo = item;
    }

    public T retirar() {
        return this.conteudo;
    }
}

// Uso
Caixa caixaTexto = new Caixa<>();
caixaTexto.guardar("Hello");
String texto = caixaTexto.retirar();

Caixa caixaNumero = new Caixa<>();
caixaNumero.guardar(42);
Integer numero = caixaNumero.retirar();
```

⚠ ⚠️ Limitação Importante: Arrays Genéricos

Java não permite criar arrays de tipos genéricos diretamente devido à **type erasure** (apagamento de tipo) em tempo de execução. O sistema de tipos genérico existe apenas em tempo de compilação.

```
//
```

Made with GAMMA

Testes e JUnit: Garantindo a Qualidade do Código

Testes unitários são fundamentais para desenvolvimento Java profissional. Eles verificam pequenas unidades de código isoladamente, garantindo que cada componente funcione corretamente antes da integração. JUnit é o framework de testes mais popular para Java, fornecendo uma estrutura robusta e anotações poderosas.

Testes Unitários

Verificam métodos e classes individuais em isolamento, usando mocks e stubs para dependências externas. São rápidos, repetíveis e focados em lógica específica.

JUnit Framework

Framework maduro e amplamente adotado com mais de 20 anos de história. Oferece anotações, asserções e suporte a diferentes versões do Java.

Qualidade Garantida

Detecta bugs precocemente, facilita refatoração com segurança e melhora a confiança no código. Reduz custos de manutenção a longo prazo.

Conceitos Fundamentais do JUnit

📌 Anotações Principais

- `@Test` - Marca método como teste
- `@BeforeEach` - Setup antes de cada teste
- `@AfterEach` - Cleanup após cada teste
- `@BeforeAll` - Setup antes de todos os testes
- `@AfterAll` - Cleanup após todos os testes
- `@DisplayName` - Nome descritivo do teste
- `@Disabled` - Desativa teste temporariamente

✅ Asserções Importantes

- `assertEquals()` - Verifica igualdade
- `assertTrue()` - Verifica condição verdadeira
- `assertFalse()` - Verifica condição falsa
- `assertNotNull()` - Verifica não-nulo
- `assertNull()` - Verifica nulo
- `assertThrows()` - Verifica exceção lançada
- `assertTimeout()` - Verifica tempo de execução

🎯 Boas Práticas

- Nome de método descreve comportamento
- Um teste, uma asserção
- Setup e cleanup automático
- Testes independentes entre si
- Coverage de casos normais e de erro
- Execução rápida e repetível
- Documentação viva do código

Estrutura de uma Classe de Teste

```java
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class CalculadoraTest {

    private Calculadora calculadora;

    @BeforeEach
    void setUp() {
        calculadora = new Calculadora();
    }

    @Test
    @DisplayName("Soma dois números positivos")
    void deveSomarDoisNumerosPositivos() {
        // Given
        int a = 5;
        int b = 3;

        // When
        int resultado = calculadora.somar(a, b);

        // Then
        assertEquals(8, resultado, "5 + 3 deve ser 8");
    }

    @Test
    @DisplayName("Soma número positivo com zero")
    void deveSomarNumeroComZero() {
        // Given
        int a = 10;
        int b = 0;

        // When
        int resultado = calculadora.somar(a, b);

        // Then
        assertEquals(10, resultado, "10 + 0 deve ser 10");
    }

    @Test
    @DisplayName("Lança exceção ao dividir por zero")
    void deveLancarExcecaoAoDividirPorZero() {
        // Given
        int a = 10;
        int b = 0;

        // When & Then
        assertThrows(ArithmeticException.class,
                () -> calculadora.dividir(a, b),
                "Divisão por zero deve lançar exceção");
    }
}
```

💡 Por que Testes são Importantes?

- **Detecta bugs precocemente** - Encontra problemas antes que cheguem à produção
- **Facilita refatoração** - Permite reescrever código com confiança de que funcionalidade está preservada
- **Documentação viva** - Exemplos de uso que sempre estão atualizados
- **Melhora design** - Código testável tende a ser mais modular e desacoplado
- **Reduz custos** - Corrigir bugs em produção é muito mais caro que detectá-los em desenvolvimento

Made with GAMMA

JUnit em Ação: Um Exemplo Prático

Vamos implementar um exemplo completo de teste JUnit para um método que soma elementos de um array. Este cenário demonstra como estruturar testes, usar asserções e cobrir diferentes casos de uso.

Cenário: Método de Soma de Array

🎯 Requisitos do Método

Implementaremos um método `somarArray` que:

- Recebe um array de inteiros como parâmetro
- Retorna a soma de todos os elementos
- Retorna 0 para arrays vazios ou nulos
- Lida corretamente com números negativos
- Funciona para arrays de qualquer tamanho

🧪 Cobertura de Testes

Nos testes, vamos cobrir:

- Caso normal - array com números positivos
- Array vazio - tamanho zero
- Array nulo - referência null
- Números negativos - mistura de positivos e negativos
- Array unitário - apenas um elemento
- Zero no array - elemento zero presente

Implementação do Método

```java
public class CalculadoraArray {

    /**
     * Soma todos os elementos de um array de inteiros.
     *
     * @param numeros Array de inteiros a serem somados
     * @return Soma de todos os elementos, ou 0 se array for null ou vazio
     */
    public int somarArray(int[] numeros) {
        // Verifica se array é null
        if (numeros == null) {
            return 0;
        }

        // Soma todos os elementos usando loop
        int soma = 0;
        for (int numero : numeros) {
            soma += numero;
        }

        return soma;
    }
}
```

Classe de Teste Completa

```java
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class CalculadoraArrayTest {

    private CalculadoraArray calculadora;

    @BeforeEach
    void setUp() {
        calculadora = new CalculadoraArray();
    }

    @Test
    @DisplayName("Soma array com números positivos")
    void deveSomarArrayComNumerosPositivos() {
        // Given
        int[] numeros = {1, 2, 3, 4, 5};

        // When
        int resultado = calculadora.somarArray(numeros);

        // Then
        assertEquals(15, resultado, "1 + 2 + 3 + 4 + 5 deve ser 15");
    }

    @Test
    @DisplayName("Soma array com números negativos e positivos")
    void deveSomarArrayComNumerosNegativos() {
        // Given
        int[] numeros = {10, -5, 3, -2};

        // When
        int resultado = calculadora.somarArray(numeros);

        // Then
        assertEquals(6, resultado, "10 - 5 + 3 - 2 deve ser 6");
    }

    @Test
    @DisplayName("Soma array vazio retorna zero")
    void deveRetornarZeroParaArrayVazio() {
        // Given
        int[] numeros = {};

        // When
        int resultado = calculadora.somarArray(numeros);

        // Then
        assertEquals(0, resultado, "Array vazio deve retornar 0");
    }

    @Test
    @DisplayName("Soma array nulo retorna zero")
    void deveRetornarZeroParaArrayNulo() {
        // Given
        int[] numeros = null;

        // When
        int resultado = calculadora.somarArray(numeros);

        // Then
        assertEquals(0, resultado, "Array nulo deve retornar 0");
    }

    @Test
    @DisplayName("Soma array com um elemento")
    void deveSomarArrayUnitario() {
        // Given
        int[] numeros = {42};

        // When
        int resultado = calculadora.somarArray(numeros);

        // Then
        assertEquals(42, resultado, "Array com um elemento deve retornar esse elemento");
    }

    @Test
    @DisplayName("Soma array com zero")
    void deveSomarArrayComZero() {
        // Given
        int[] numeros = {1, 0, 3};

        // When
        int resultado = calculadora.somarArray(numeros);

        // Then
        assertEquals(4, resultado, "1 + 0 + 3 deve ser 4");
    }
}
```

Resultados dos Testes

6	100%	0
Testes Implementados	**Coverage de Casos**	**Bugs Encontrados**
Cobertura completa de cenários principais	Normal, edge cases e casos de erro	Implementação correta desde o início

✅ Execução de Testes Bem-Sucedida

Todos os testes passaram! A implementação do método `somarArray` está correta e robusta. A cobertura inclui casos normais, edge cases (array vazio, nulo, unitário) e casos especiais (números negativos, zero no array).

Output do JUnit:

```
JUnit Jupiter: 6 tests successful
  ✓ deveSomarArrayComNumerosPositivos
  ✓ deveSomarArrayComNumerosNegativos
  ✓ deveRetornarZeroParaArrayVazio
  ✓ deveRetornarZeroParaArrayNulo
  ✓ deveSomarArrayUnitario
  ✓ deveSomarArrayComZero
```

Aprenda com Este Exemplo

Estruture Testes

Use `@BeforeEach` para setup e organize métodos de forma lógica

Cobre Cenários

Testes normais, edge cases e casos de erro garantem robustez

Nome Claro

`@DisplayName` torna intenção do teste óbvia para outros desenvolvedores

Made with GAMMA

Recursão: Conceitos, Tipos e Boas Práticas

A recursão é uma técnica poderosa em programação onde uma função chama a si mesma para resolver um problema. É fundamental para algoritmos que processam estruturas de dados naturalmente recursivas, como árvores e grafos.

O Que é Recursão?

Uma função recursiva resolve um problema quebrando-o em subproblemas menores da mesma natureza, até atingir um **caso base** que pode ser resolvido diretamente sem mais chamadas recursivas. Sem um caso base bem definido, a recursão resultará em um erro de estouro de pilha (StackOverflowError).

Tipos Comuns de Recursão

1

Recursão Direta

Uma função chama a si mesma diretamente.

```
int fatorial(int n) {
    if (n == 0) return 1;
    return n * fatorial(n - 1);
}
```

2

Recursão Indireta

Duas ou mais funções se chamam mutuamente em um ciclo.

```
void funcA(int n) {
    if (n > 0) funcB(n - 1);
}
void funcB(int n) {
    if (n > 0) funcA(n - 1);
}
```

3

Recursão de Cauda

A chamada recursiva é a última operação na função, permitindo otimização em algumas linguagens (não nativa em Java).

```
int tailFatorial(int n, int acc) {
    if (n == 0) return acc;
    return tailFatorial(n - 1, acc * n);
}
```

Boas Práticas para Recursão em Java

1 **Defina um Caso Base Claro**

É a condição de parada essencial para evitar loops infinitos e `StackOverflowError`.

2 **Garanta a Progressão**

Cada chamada recursiva deve se aproximar do caso base, resolvendo um subproblema menor.

3 **Considere a Performance**

A recursão pode gerar overhead devido às chamadas de função. Em Java, para problemas com muitas iterações, prefira soluções iterativas para evitar `StackOverflowError`.

4 **Priorize a Clareza**

Use recursão quando ela tornar a lógica do código mais simples e intuitiva, especialmente com estruturas de dados recursivas.

Made with GAMMA